COMP102 Lecture 7: Graphs I: Representation

Jalal Maaouni
May 21, 2026

Plan

1. Graphs as Models
2. Direction and Weight
3. Adjacency Lists
4. Adjacency Matrices
5. Representation Tradeoffs
6. Modeling a Graph
7. What Comes Next

Graphs as Models

Graphs are everywhere

Graphs are everywhere

A graph models relationships

A graph is useful when the important information is not only the objects, but **how those objects are connected**.

Graphs are everywhere

A graph models relationships

A graph is useful when the important information is not only the objects, but **how those objects are connected**.

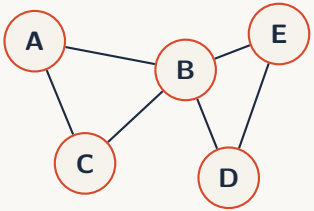
Situation	Vertices	Edges
Road map	cities	roads
Social network	people	follows/friendships
Web	pages	links
Code base	files	imports
Courses	courses	prerequisites

Graphs are everywhere

A graph models relationships

A graph is useful when the important information is not only the objects, but **how those objects are connected**.

Situation	Vertices	Edges
Road map	cities	roads
Social network	people	follows/friendships
Web	pages	links
Code base	files	imports
Courses	courses	prerequisites

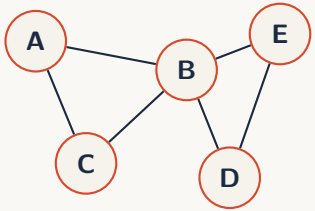


Graphs are everywhere

A graph models relationships

A graph is useful when the important information is not only the objects, but **how those objects are connected**.

Situation	Vertices	Edges
Road map	cities	roads
Social network	people	follows/friendships
Web	pages	links
Code base	files	imports
Courses	courses	prerequisites



The same drawing can represent roads, friends, webpages, dependencies, or game states. The model decides what nodes and edges mean.

What is a graph?

Definition 1.1 Graph

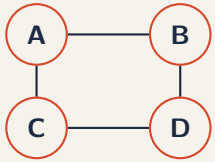
A graph is a pair $G = (V, E)$ where V is a set of **vertices** and E is a set of **edges** connecting vertices.

What is a graph?

Definition 1.2 Graph

A graph is a pair $G = (V, E)$ where V is a set of **vertices** and E is a set of **edges** connecting vertices.

Undirected example

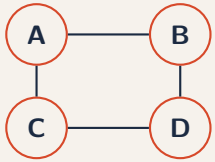


What is a graph?

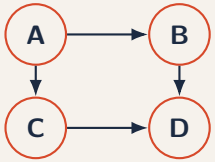
Definition 1.3 Graph

A graph is a pair $G = (V, E)$ where V is a set of **vertices** and E is a set of **edges** connecting vertices.

Undirected example



Directed example

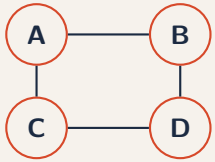


What is a graph?

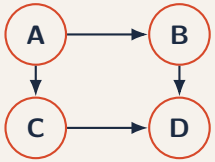
Definition 1.4 Graph

A graph is a pair $G = (V, E)$ where V is a set of **vertices** and E is a set of **edges** connecting vertices.

Undirected example



Directed example

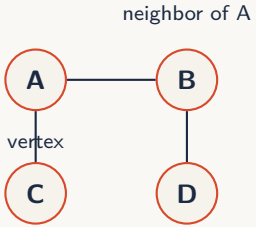


Vocabulary: vertex, edge, neighbor, degree, path, direction, and weight.

Vocabulary I: vertices, edges, neighbors

Words

- A **vertex** is an object.
- An **edge** is a connection.
- A **neighbor** is directly connected to a vertex.

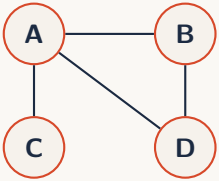


In this graph, the neighbors of A are B and C.

Vocabulary II: degree

Definition 1.5 Degree

The **degree** of a vertex is the number of edges touching it.



Example

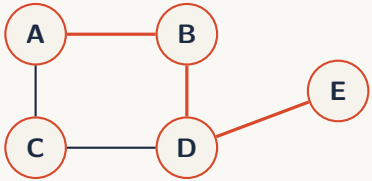
- $\deg(A) = 3$
- $\deg(B) = 2$
- $\deg(C) = 1$
- $\deg(D) = 2$

Degree measures how connected a vertex is.

Vocabulary III: paths

Definition 1.6 Path

A **path** is a sequence of vertices where each consecutive pair is connected by an edge.



Example

$$A \to B \to D \to E$$

is a path.

Length

The length of this path is 3 edges.

Vocabulary IV: direction

Undirected edge

$$A - B$$

means:

A connected to B

and

B connected to A

symmetric relation

Directed edge

$$A \to B$$

means:

A points to B

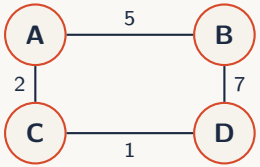
but not necessarily the reverse.

asymmetric relation

Vocabulary V: weights

Definition 1.7 Weight

A **weight** is a value attached to an edge.



Weights can mean

- distance;
- time;
- cost;
- capacity;
- difficulty.

The meaning of a weight comes from the model, not from the graph itself.

Modeling relations

Modeling relations

The hard question

Before choosing code, choose the model:

- What are the vertices?
- What are the edges?
- Are edges directed?
- Do edges need weights?

The hard question

Before choosing code, choose the model:

- What are the vertices?
- What are the edges?
- Are edges directed?
- Do edges need weights?

Remark 1.1. Modeling depends on the question

A city map can use distance, travel time, toll cost, or safety as the edge weight.

The hard question

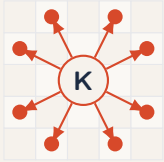
Before choosing code, choose the model:

- What are the vertices?
- What are the edges?
- Are edges directed?
- Do edges need weights?

Remark 1.1. Modeling depends on the question

A city map can use distance, travel time, toll cost, or safety as the edge weight.

Example: knight moves

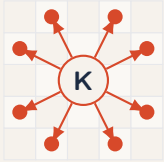


The hard question

Before choosing code, choose the model:

- What are the vertices?
- What are the edges?
- Are edges directed?
- Do edges need weights?

Example: knight moves



Remark 1.1. Modeling depends on the question

A city map can use distance, travel time, toll cost, or safety as the edge weight.

Vertices can be objects, positions, configurations, or states. Graph modeling is a translation from a problem to structure.

Direction and Weight

Directed vs undirected

Directed vs undirected

Undirected relation

If A is connected to B, then B is connected to A.

Directed vs undirected

Undirected relation

If A is connected to B, then B is connected to A.

friendship / two-way road

Directed vs undirected

Undirected relation

If A is connected to B, then B is connected to A.

friendship / two-way road

Directed relation

If A points to B, B does not necessarily point to A.

Directed vs undirected

Undirected relation

If A is connected to B, then B is connected to A.

friendship / two-way road

Directed relation

If A points to B, B does not necessarily point to A.

follow / link / prerequisite

Directed vs undirected

Undirected relation

If A is connected to B, then B is connected to A.

friendship / two-way road

Directed relation

If A points to B, B does not necessarily point to A.

follow / link / prerequisite

For an undirected edge, store both directions. For a directed edge, store only the direction that exists.

Weighted vs unweighted

Weighted vs unweighted

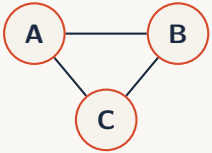
Unweighted graph

All edges are treated equally.

Weighted vs unweighted

Unweighted graph

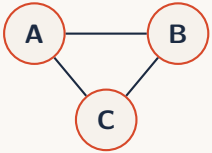
All edges are treated equally.



Weighted vs unweighted

Unweighted graph

All edges are treated equally.



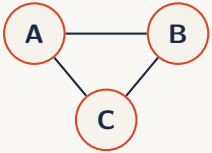
Weighted graph

Each edge carries a value: cost, distance, time, capacity, risk, or probability.

Weighted vs unweighted

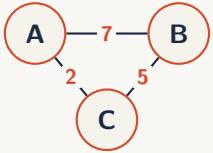
Unweighted graph

All edges are treated equally.



Weighted graph

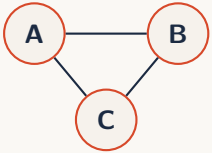
Each edge carries a value: cost, distance, time, capacity, risk, or probability.



Weighted vs unweighted

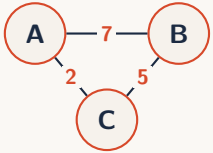
Unweighted graph

All edges are treated equally.



Weighted graph

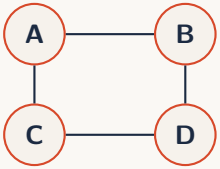
Each edge carries a value: cost, distance, time, capacity, risk, or probability.



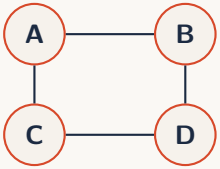
A weight is not automatically a distance. Its meaning comes from the problem model.

Adjacency Lists

Adjacency list



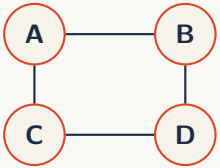
Adjacency list



Idea

For every vertex, store the list of its neighbors.

Adjacency list

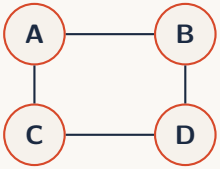


Idea

For every vertex, store the list of its neighbors.

Vertex	Neighbors
A	B, C
B	A, D
C	A, D
D	B, C

Adjacency list



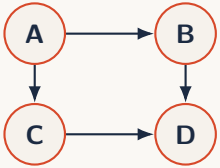
Idea

For every vertex, store the list of its neighbors.

Vertex	Neighbors
A	B, C
B	A, D
C	A, D
D	B, C

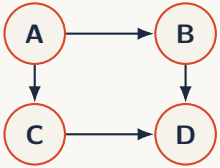
Adjacency lists are the default representation for many graph problems because they store only existing connections.

Directed adjacency lists




```
graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": [],
}
```

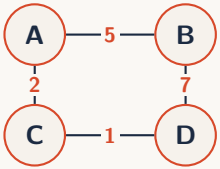
Directed adjacency lists




```
graph = {
    "A": ["B", "C"],
    "B": ["D"],
    "C": ["D"],
    "D": [],
}
```

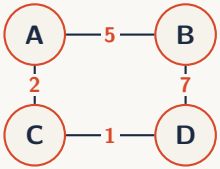
For a directed edge `A -> B`, store `B` in `A`'s list. Do not automatically store `A` in `B`'s list.

Weighted adjacency lists




```python
graph = {
    "A": [("B", 5), ("C", 2)],
    "B": [("A", 5), ("D", 7)],
    "C": [("A", 2), ("D", 1)],
    "D": [("B", 7), ("C", 1)],
}
```

Weighted adjacency lists

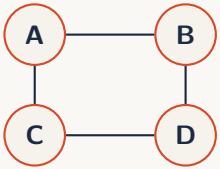



```python
graph = {
    "A": [("B", 5), ("C", 2)],
    "B": [("A", 5), ("D", 7)],
    "C": [("A", 2), ("D", 1)],
    "D": [("B", 7), ("C", 1)],
}
```

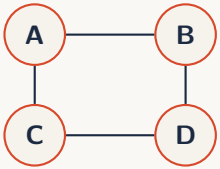
A weighted adjacency list stores extra information with each neighbor: usually `(neighbor, weight)`.

Adjacency Matrices

Adjacency matrix



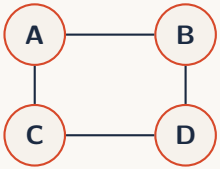
Adjacency matrix



Idea

Use a 2D table. Entry (i, j) says whether vertex i is connected to vertex j.

Adjacency matrix

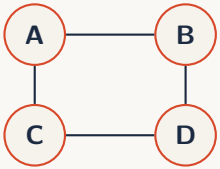


Idea

Use a 2D table. Entry (i, j) says whether vertex i is connected to vertex j.

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Adjacency matrix



Idea

Use a 2D table. Entry (i, j) says whether vertex i is connected to vertex j.

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

For an undirected graph, the adjacency matrix is symmetric.

Directed and weighted matrices

Directed and weighted matrices

Directed matrix

Direction breaks symmetry.

Directed and weighted matrices

Directed matrix

Direction breaks symmetry.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

Directed and weighted matrices

Directed matrix

Direction breaks symmetry.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

Weighted matrix

Entries store weights, not only 0 or 1.

Directed and weighted matrices

Directed matrix

Direction breaks symmetry.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

Weighted matrix

Entries store weights, not only 0 or 1.

	A	B	C	D
A	0	5	2	INF
B	5	0	INF	7
C	2	INF	0	1
D	INF	7	1	0

Directed and weighted matrices

Directed matrix

Direction breaks symmetry.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0

Weighted matrix

Entries store weights, not only 0 or 1.

	A	B	C	D
A	0	5	2	INF
B	5	0	INF	7
C	2	INF	0	1
D	INF	7	1	0

Do not use `0` for “no edge” if zero-weight edges are possible. Use `INF` or `None` depending on the algorithm.

Representation Tradeoffs

List vs matrix

Let $n = |V|$ be the number of vertices and $m = |E|$ the number of edges.

Operation	**Adjacency list**	**Adjacency matrix**
Memory	$O(n + m)$	$O(n^2)$
Check edge $u \to v$	$O(\deg(u))$	$O(1)$
Iterate neighbors of u	$O(\deg(u))$	$O(n)$
Add edge	usually $O(1)$	$O(1)$
Remove edge	depends on structure	$O(1)$

List vs matrix

Let $n = |V|$ be the number of vertices and $m = |E|$ the number of edges.

Operation	Adjacency list	Adjacency matrix
Memory	$O(n + m)$	$O(n^2)$
Check edge $u \to v$	$O(\deg(u))$	$O(1)$
Iterate neighbors of u	$O(\deg(u))$	$O(n)$
Add edge	usually $O(1)$	$O(1)$
Remove edge	depends on structure	$O(1)$

Adjacency lists are neighbor-friendly and memory-efficient. Matrices are edge-lookup-friendly but pay $O(n^2)$ memory.

Sparse vs dense graphs

Sparse vs dense graphs

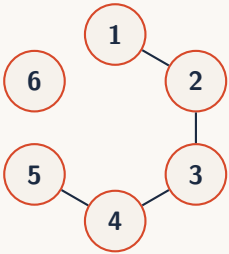
Sparse graph

$m \ll n^2$. Most possible edges are absent.

Sparse vs dense graphs

Sparse graph

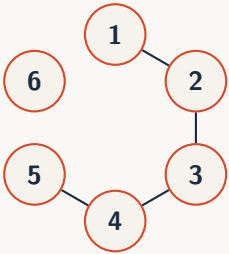
$m \ll n^2$. Most possible edges are absent.



Sparse vs dense graphs

Sparse graph

$m \ll n^2$. Most possible edges are absent.



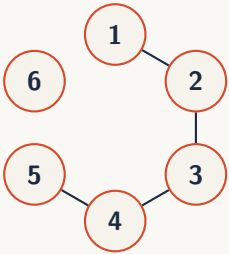
Dense graph

m is close to n^2. Many possible edges exist.

Sparse vs dense graphs

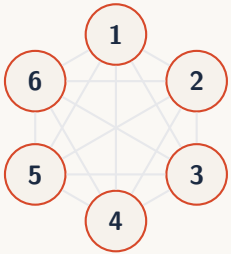
Sparse graph

$m \ll n^2$. Most possible edges are absent.



Dense graph

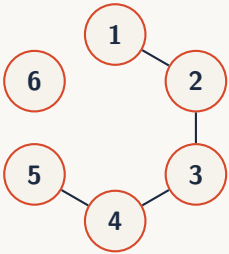
m is close to n^2. Many possible edges exist.



Sparse vs dense graphs

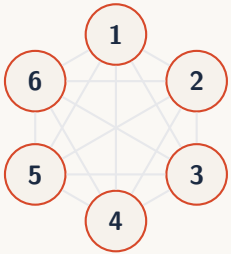
Sparse graph

$m \ll n^2$. Most possible edges are absent.



Dense graph

m is close to n^2. Many possible edges exist.



Most real-world graphs are sparse: roads, imports, prerequisites, and many social or web graphs do not contain all possible connections.

Modeling a Graph

Coding: Build a graph

```python
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

```python
def build_undirected_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    return graph
```

Coding: Build a graph

Goal

Convert a list of input edges into an adjacency list.

```
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

```
def build_undirected_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    return graph
```

Coding: Build a graph

Goal

Convert a list of input edges into an adjacency list.

```
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

```
def build_undirected_graph(vertices, edges):
    graph = {v: [] for v in vertices}

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    return graph
```

The initialization step is not optional: it preserves isolated vertices.

Coding: Build an adjacency matrix

```
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

```
def build_undirected_matrix(vertices, edges):
    n = len(vertices)
    index = {v: i for i, v in enumerate(vertices)}

    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        i = index[u]
        j = index[v]
        matrix[i][j] = 1
        matrix[j][i] = 1

    return matrix
```

Coding: Build an adjacency matrix

Goal

Convert a list of input edges into a 0/1 adjacency matrix.

```
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

```
def build_undirected_matrix(vertices, edges):
    n = len(vertices)
    index = {v: i for i, v in enumerate(vertices)}

    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        i = index[u]
        j = index[v]
        matrix[i][j] = 1
        matrix[j][i] = 1

    return matrix
```

Coding: Build an adjacency matrix

Goal

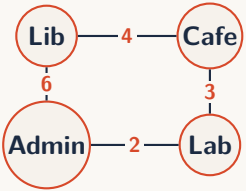
Convert a list of input edges into a 0/1 adjacency matrix.

```
vertices = ["A", "B", "C", "D"]
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

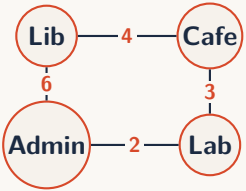
```
def build_undirected_matrix(vertices, edges):
    n = len(vertices)
    index = {v: i for i, v in enumerate(vertices)}

    matrix = [[0] * n for _ in range(n)]

    for u, v in edges:
        i = index[u]
        j = index[v]
        matrix[i][j] = 1
        matrix[j][i] = 1

    return matrix
```

The dictionary `index` converts vertex names into matrix positions.

Case study: campus map



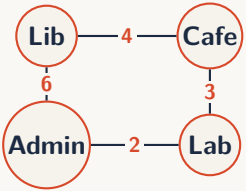
Case study: campus map



Questions

- What are the vertices?
- What are the edges?
- Is the graph directed?
- Should the graph be weighted?
- What could the weights mean?

Case study: campus map



Questions

- What are the vertices?
- What are the edges?
- Is the graph directed?
- Should the graph be weighted?
- What could the weights mean?

Remark 6.1. Representation choice

For a campus map, an adjacency list is usually natural because each place connects to only a few other places.

What Comes Next

What comes next

What comes next

Today

We learned how to:

- model relations as graphs;
- distinguish direction and weight;
- store graphs using lists and matrices;
- choose a representation based on tradeoffs.

What comes next

Today

We learned how to:

- model relations as graphs;
- distinguish direction and weight;
- store graphs using lists and matrices;
- choose a representation based on tradeoffs.

Next lecture

Once a graph is represented, we can start asking algorithmic questions:

- Can we reach one vertex from another?
- What order should we explore vertices in?
- What is the shortest path?
- Is the graph connected?